12/10/2026

AGENTS

Artificial Intelligence Lab 06

Instructor: Sir Mustajab

Submitted by: Wasif Ali

Reg id: 245187

Class: BSCS F24 B



Lab No 06

Lab Task 1:

1. *(Easy)* **Implement a table-driven agent** in a Python script `traffic_light_agent.py` for a traffic light responder. The percept is the current light color (`"Red"`, `"Yellow"`, or `"Green"`); store a lookup table (dictionary) mapping each color directly to an action, and a function `traffic_light_agent(percept)` that returns the table's action for that percept. Test it against a sequence of at least 4 percepts.

   Python – Input:

   ```
   table = {"Red": "Stop", "Yellow": "Slow Down", "Green": "Go"}
   percept_sequence = ["Red", "Green", "Yellow", "Red"]

   Sample Output:
   Percept Action
   Red Stop
   Green Go
   Yellow Slow Down
   Red Stop
   ```

   Mandatory Git/GitHub steps:

   (a) `git init` a repository named `traffic-light-agent`; commit the first working version with `git add` and `git commit`.

   (b) Add a new entry to the table for a `"Flashing Red"` percept (action: `"Stop and Proceed with Caution"`); use `git diff` to review the change before staging it, then commit it.

   (c) Create a GitHub repository named `traffic-light-agent` and push your repository to it with `git remote add origin` and `git push`.

Code:

```python
table = {
    "Red": "Stop",
    "Yellow": "Slow Down",
    "Green": "Go",
}

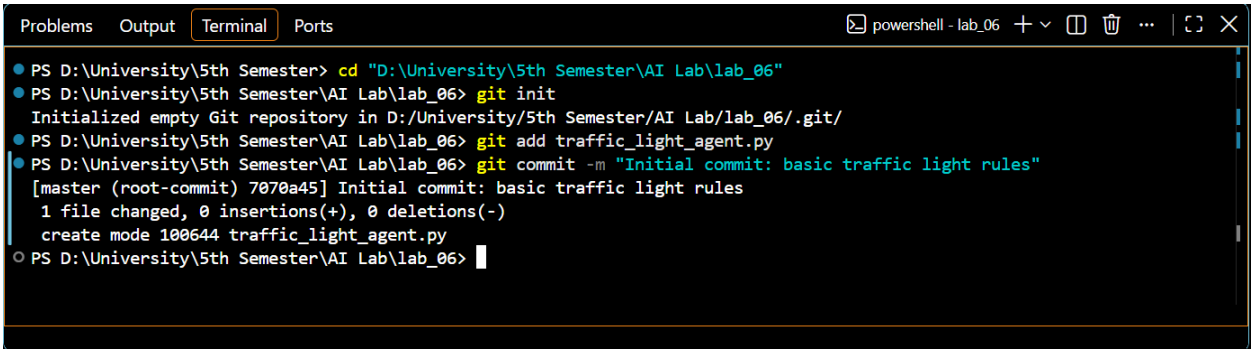
percept_sequence = ["Red", "Green", "Yellow", "Red"]

def traffic_light_agent(percept):
    return table.get(percept)

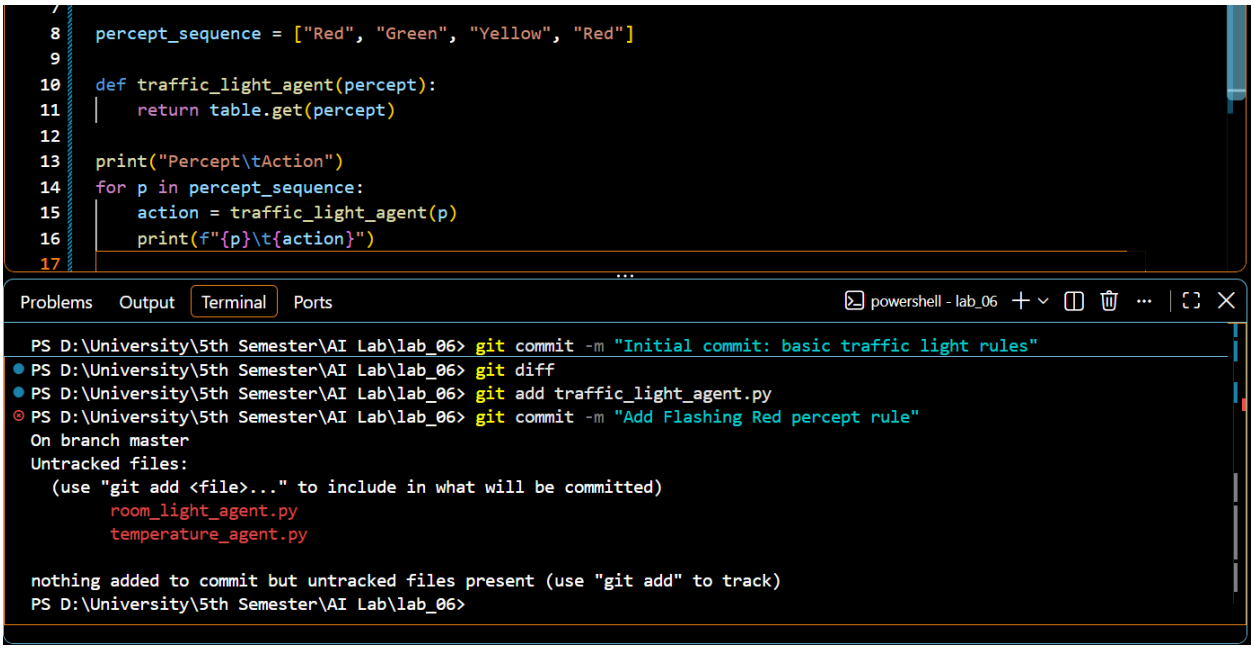
print("Percept\tAction")
for p in percept_sequence:
    action = traffic_light_agent(p)
    print(f"{p}\t{action}")
```

Git Commands

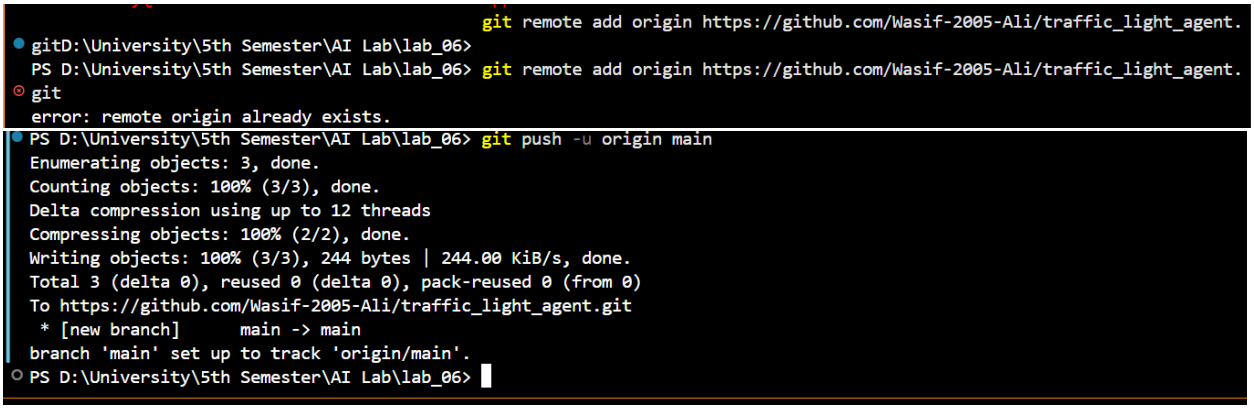
Initialize and commit the first version:



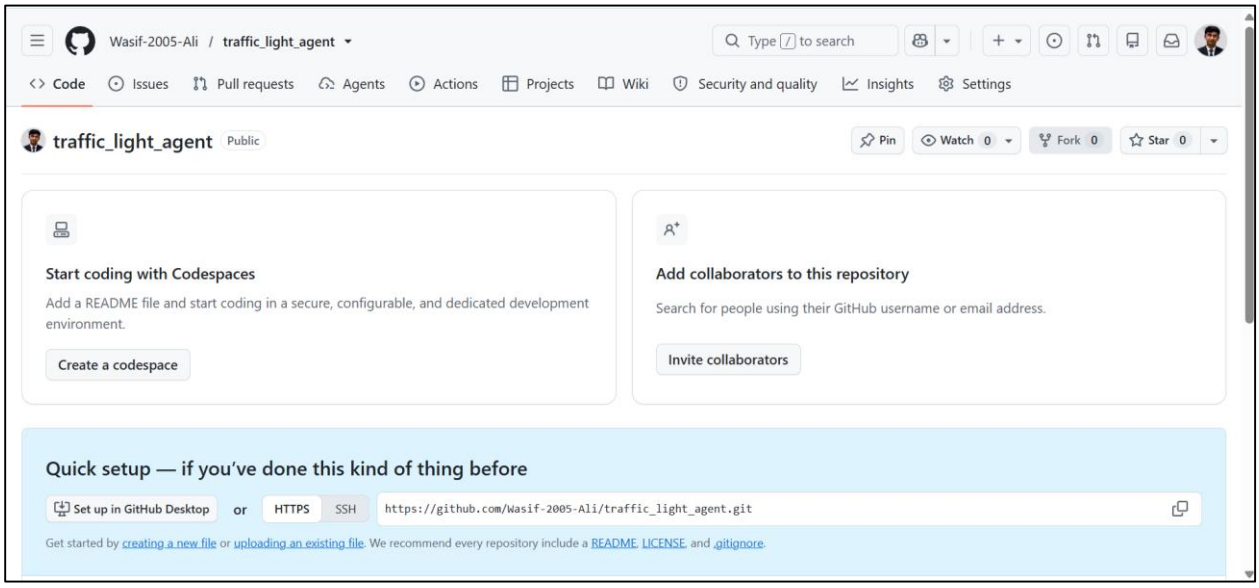
Add the "Flashing Red" entry to the script, review with git diff, then stage and commit:

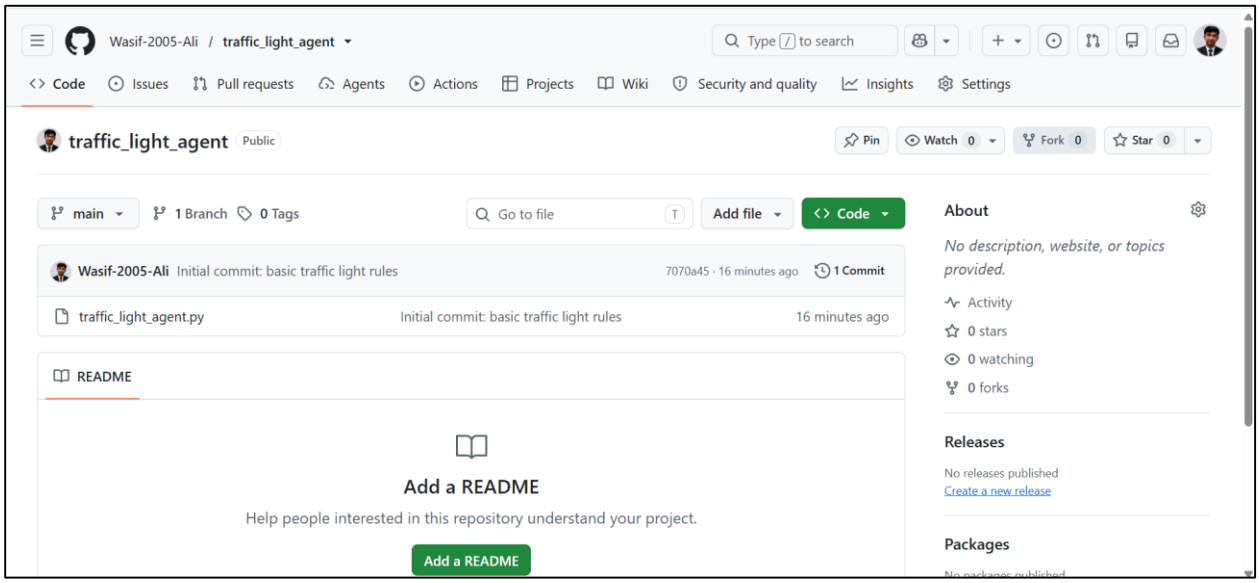


Push to GitHub:



GITHUB VIEW:


Wasif-2005-Ali / traffic_light_agent
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
traffic_light_agent
Public
Pin
Watch 0
Fork 0
Star 0
Start coding with Codespaces
Add a README file and start coding in a secure, configurable, and dedicated development environment.
Create a codespace
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in GitHub Desktop
or
HTTPS
SSH
https://github.com/Wasif-2005-Ali/traffic_light_agent.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.


Wasif-2005-Ali / traffic_light_agent
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
traffic_light_agent
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
T
Add file
Code
About
No description, website, or topics provided.
Wasif-2005-Ali Initial commit: basic traffic light rules
7070a45 · 16 minutes ago
1 Commit
traffic_light_agent.py
Initial commit: basic traffic light rules
16 minutes ago
Activity
0 stars
0 watching
0 forks
README
Releases
No releases published
Create a new release
Add a README
Help people interested in this repository understand your project.
Add a README
Packages

Lab Task 2:

2. *(Easy)* **Implement a simple reflex agent** in a Python script `room_light_agent.py` for a smart room light. The percept is the room's brightness (`"Dark"`, `"Dim"`, or `"Bright"`); using `if/elif/else` condition-action rules (not a table), write a function `room_light_agent(percept)` that returns the correct action. Test it against a sequence of at least 4 percepts.

Python – Input:

```
percept_sequence = ["Dark", "Dim", "Bright", "Dark"]

Rules:
Dark -> "Turn Light ON"
Bright -> "Turn Light OFF"
Dim -> "Leave Light As Is"

Sample Output:
Percept Action
Dark Turn Light ON
Dim Leave Light As Is
Bright Turn Light OFF
Dark Turn Light ON
```

Mandatory Git/GitHub steps:

(a) `git init` a repository named `room-light-agent`; commit the first working version with `git add` and `git commit`.

(b) Run `git status` after editing the file to add a rule for a `"Very Bright"` percept, then commit the change.

(c) Create a GitHub repository named `room-light-agent` and push your repository to it with `git push`.

Code:

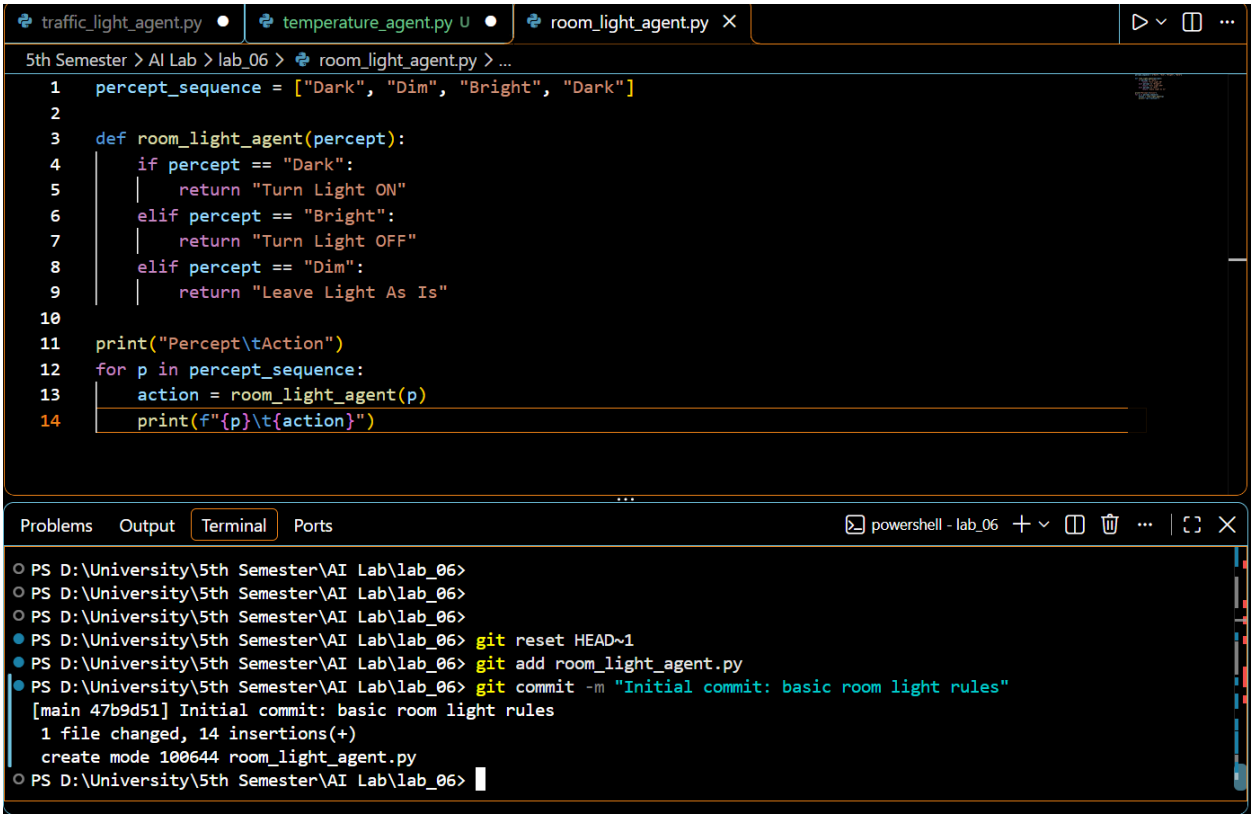
```python
percept_sequence = ["Dark", "Dim", "Bright", "Dark"]

def room_light_agent(percept):
    if percept == "Dark":
        return "Turn Light ON"
    elif percept == "Bright":
        return "Turn Light OFF"
    elif percept == "Dim":
        return "Leave Light As Is"
    elif percept == "Very Bright":
        return "Turn Light OFF completely"

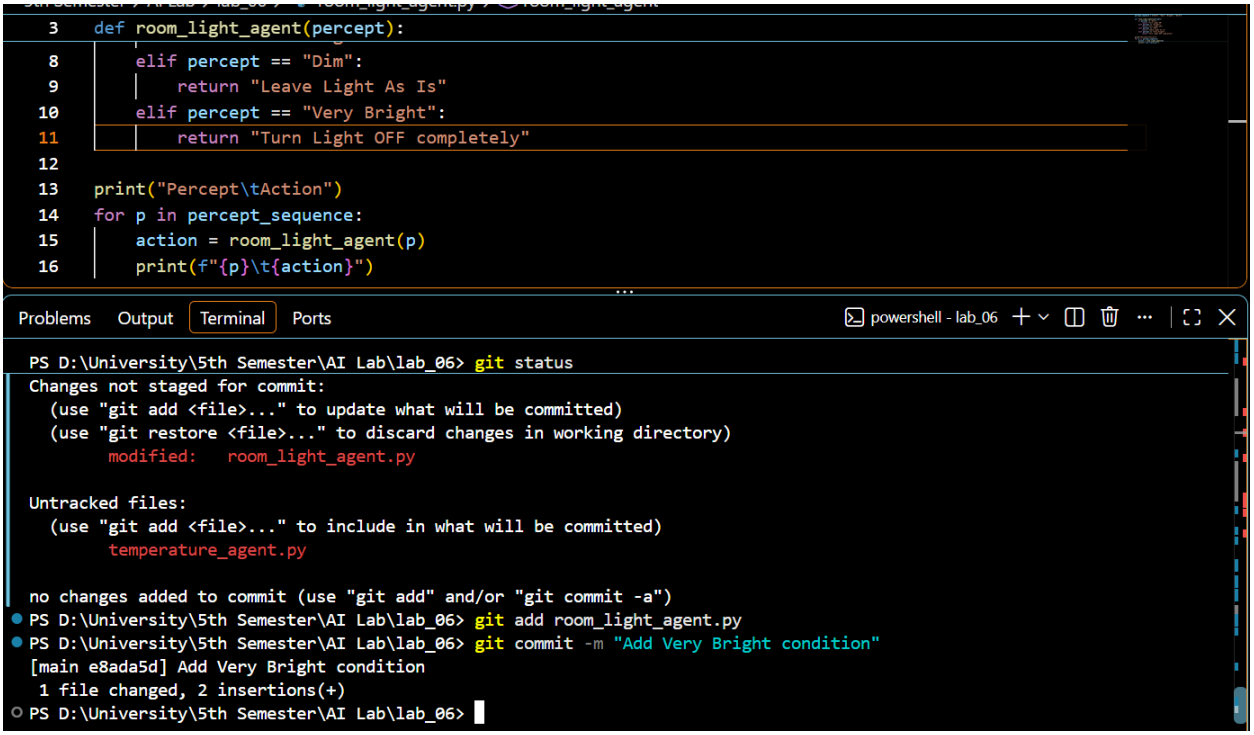
print("Percept\tAction")
for p in percept_sequence:
    action = room_light_agent(p)
    print(f"{p}\t{action}")
```

Git Commands

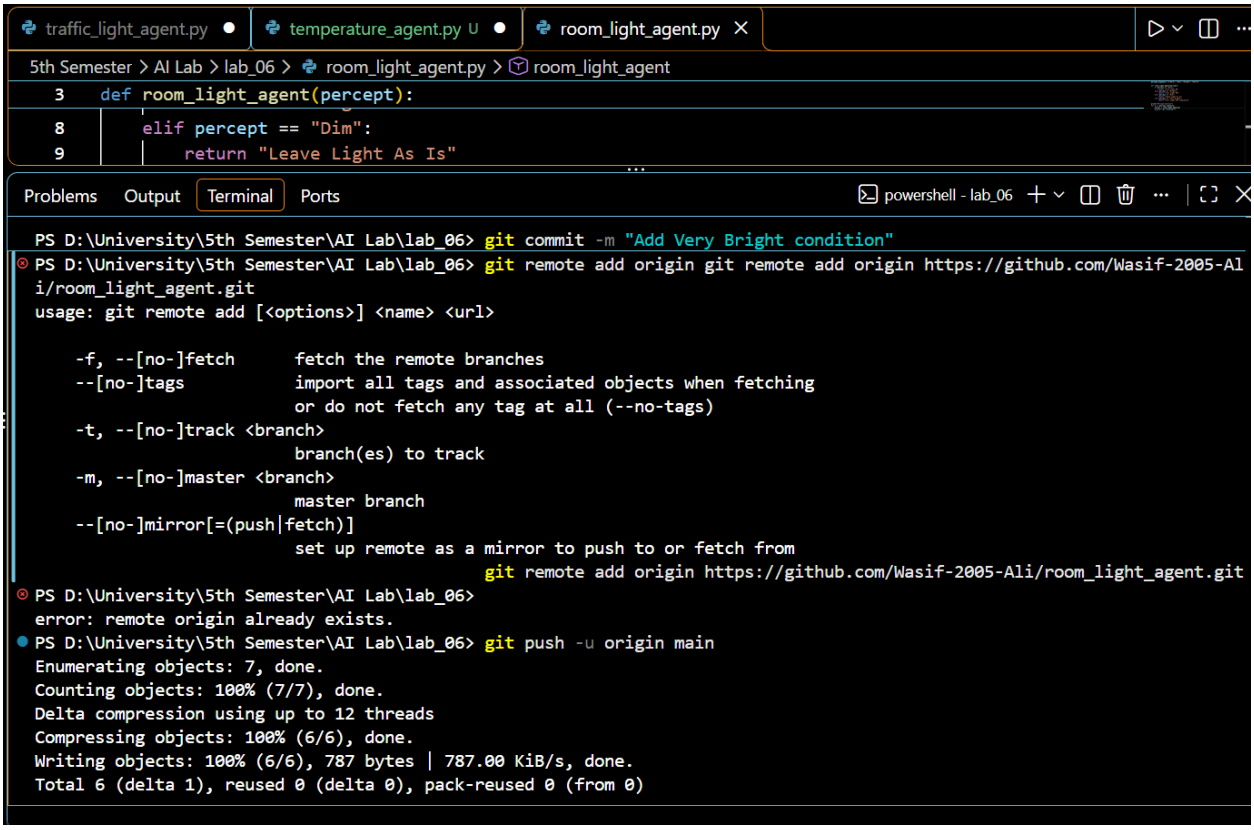
Initialize and commit the first version:



Add the "Very Bright" rule to the code, check status, and commit



Push to GitHub



Lab Task 3:

3. *(Easy)* **Implement a model-based reflex agent** in a Python script `temperature_agent.py` that manages the heater for two rooms, `"Room1"` and `"Room2"`. The percept is `(room, reading)` where `reading` is `"Cold"`, `"Normal"`, or `"Hot"`, and only one room is perceived at a time. Keep an internal dictionary `model` remembering the last known reading of *each* room, and write `temperature_agent(room, reading)` that: turns the heater ON if the current room is Cold, OFF if Hot, reports `"All Rooms Comfortable - Idle"` once the model shows *both* rooms Normal, and otherwise reports `"Move to Other Room to Check"`. Also keep a dictionary counting how many times the heater was turned on per room.

Python — Input:

```
percept_sequence = [("Room1", "Cold"), ("Room1", "Normal"),
                    ("Room2", "Hot"), ("Room2", "Normal")]

Sample Output:
Room Reading Action Model
Room1 Cold Turn Heater ON {'Room1': 'Cold', 'Room2': None}
Room1 Normal Move to Other Room to Check {'Room1': 'Normal', 'Room2': None}
Room2 Hot Turn Heater OFF {'Room1': 'Normal', 'Room2': 'Hot'}
Room2 Normal All Rooms Comfortable- Idle {'Room1': 'Normal', 'Room2': '
    Normal'}

Heater ON count: {'Room1': 1, 'Room2': 0}
```

Mandatory Git/GitHub steps:

(a) `git init` a repository named `temperature-agent`; commit the first working version with `git add` and `git commit`.

(b) Create and switch to a branch named `add-heater-count`, where you add the heater ON-count tracking; commit this change on that branch, then switch back to `main` and `git merge` it in.

(c) Create a GitHub repository named `temperature-agent`, push your repository with `git push`, then `git clone` it into a second local folder to confirm it downloads correctly, and `git pull` in your original folder after making one more small change from the clone.

```python
model = {'Room1': None, 'Room2': None}

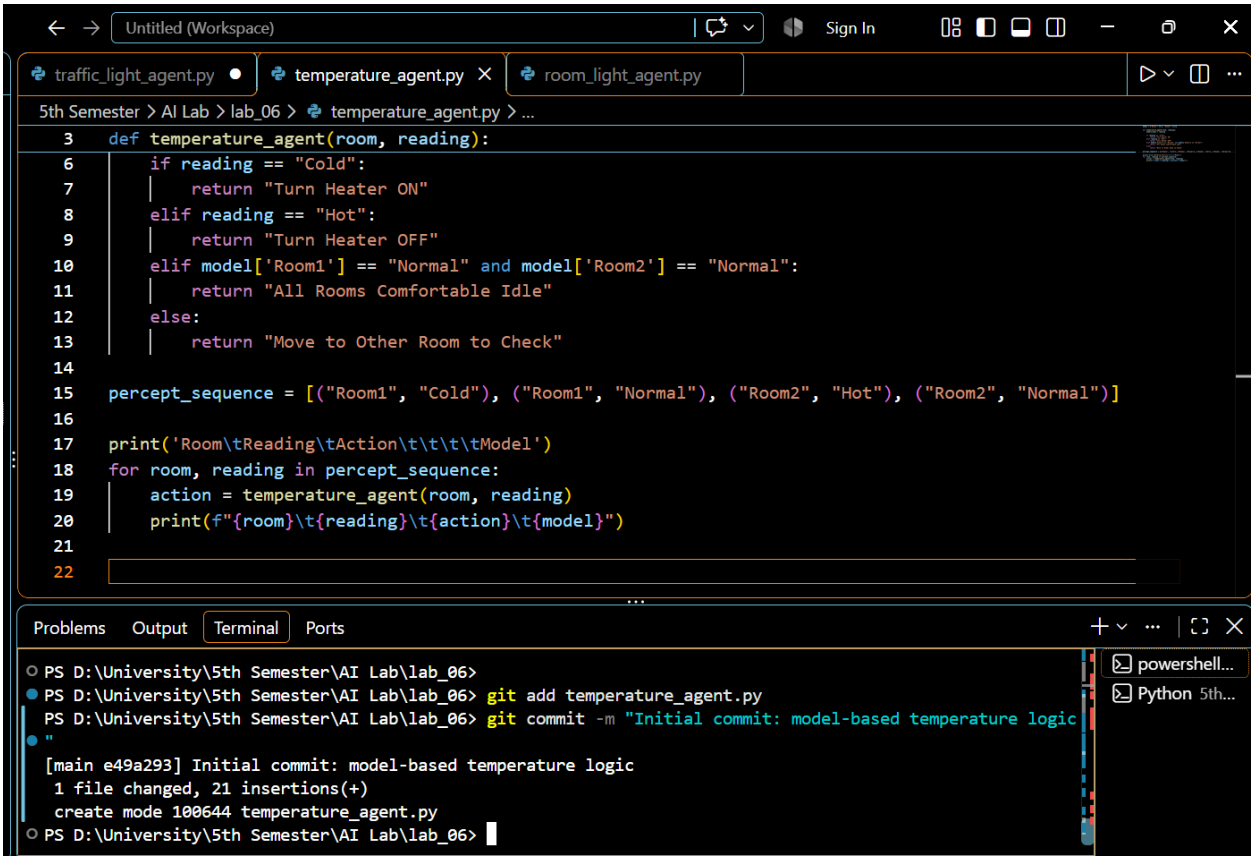
def temperature_agent(room, reading):
    model[room] = reading

    if reading == "Cold":
        return "Turn Heater ON"
    elif reading == "Hot":
        return "Turn Heater OFF"
    elif model['Room1'] == "Normal" and model['Room2'] == "Normal":
        return "All Rooms Comfortable Idle"
    else:
        return "Move to Other Room to Check"

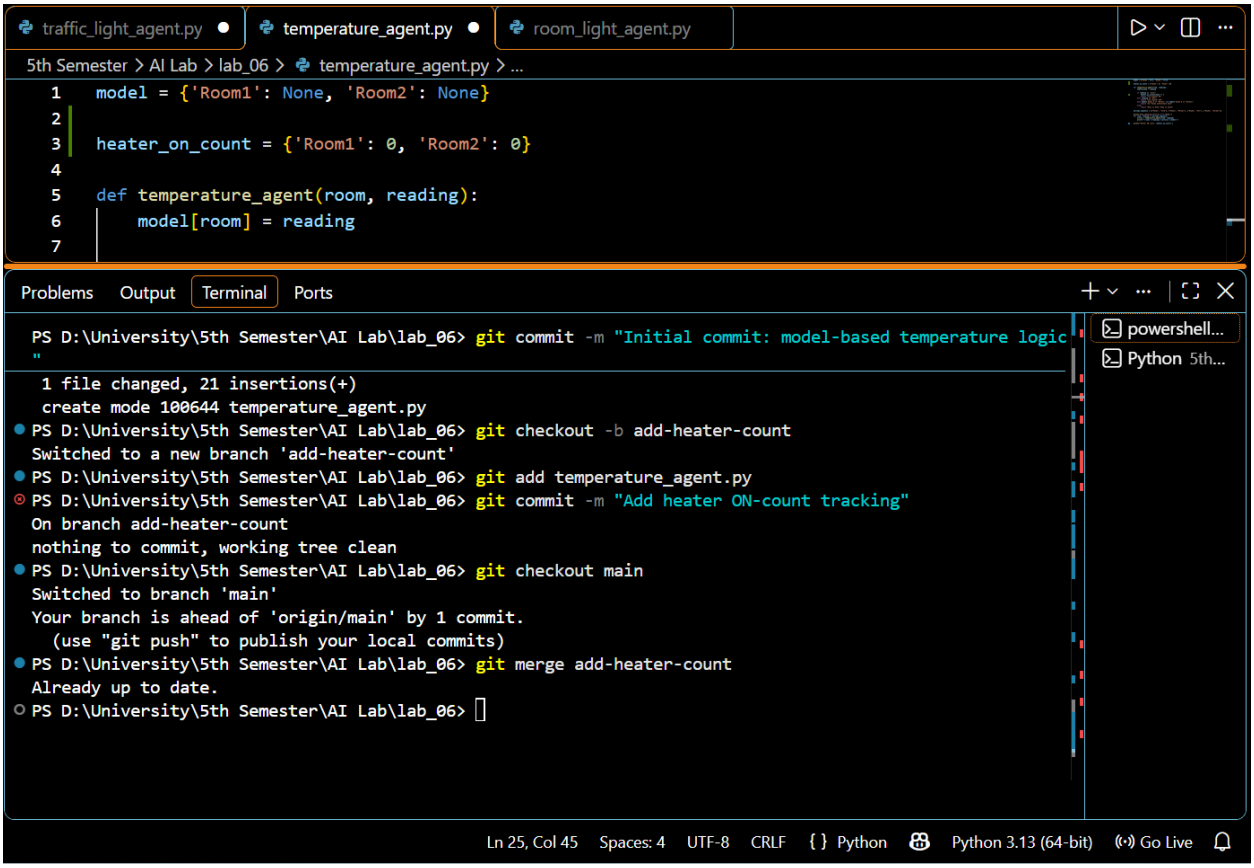
percept_sequence = [("Room1", "Cold"), ("Room1", "Normal"), ("Room2", "Hot"),
("Room2", "Normal")]

print('Room\tReading\tAction\t\t\t\tModel')
for room, reading in percept_sequence:
    action = temperature_agent(room, reading)
    print(f"{room}\t{reading}\t{action}\t{model}")
```

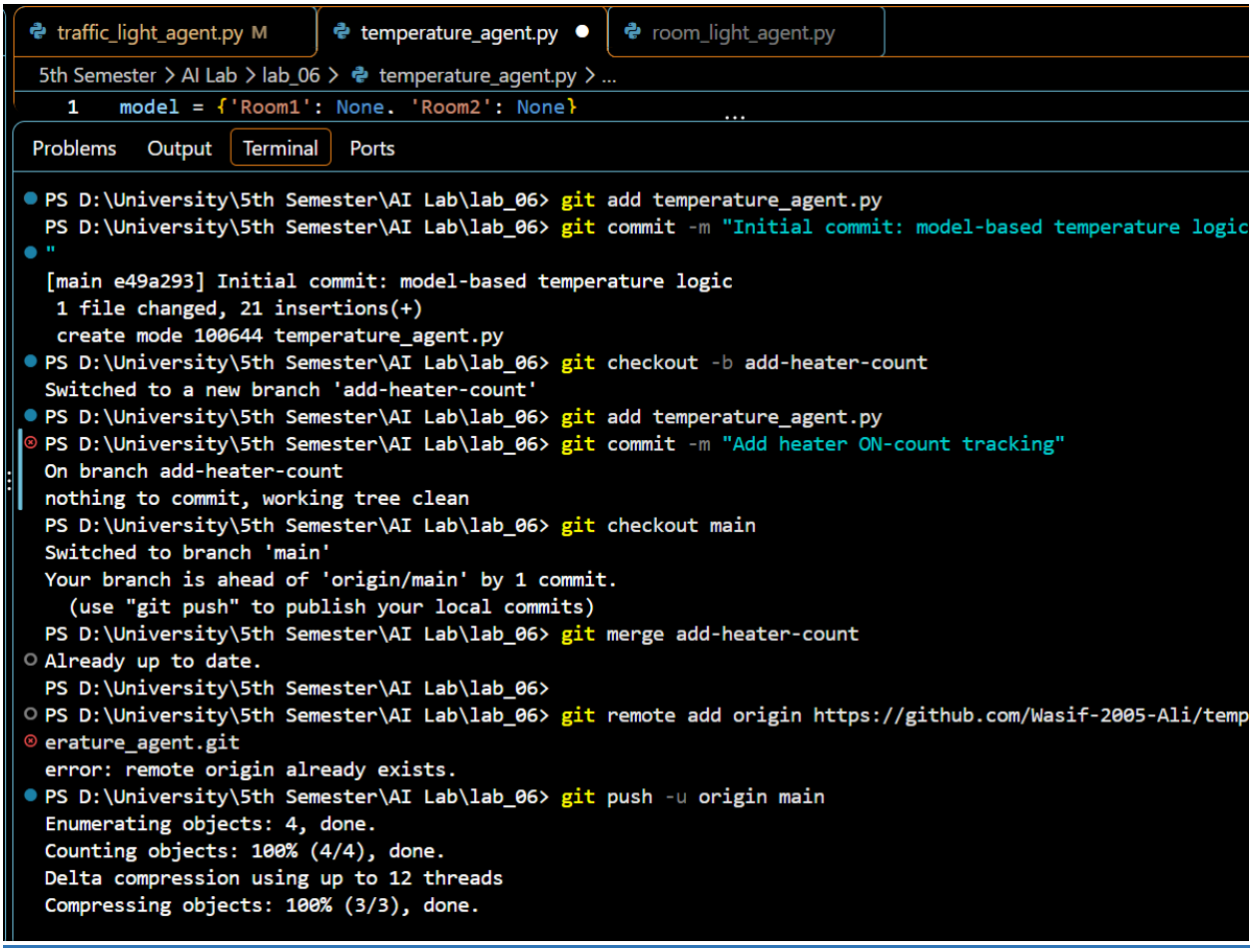
Git Commands: (Initialize and commit the first version (without the heater count))

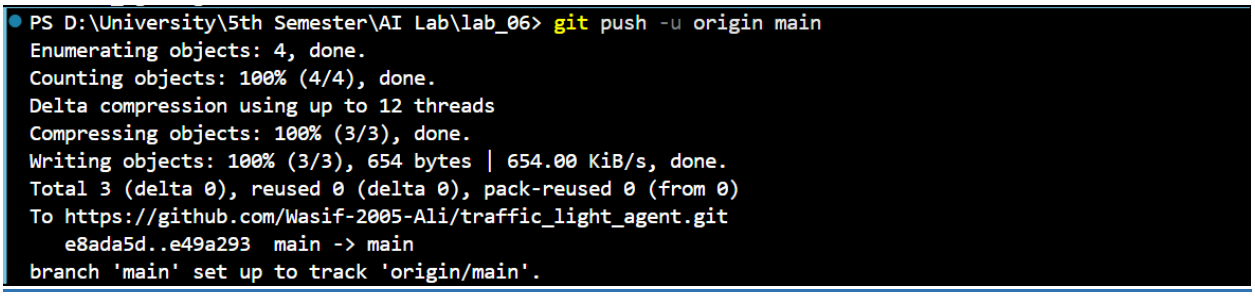


Create the branch, add heater counting logic in the code, commit, and merge back to main:



Push, clone into a second folder, and pull updates:





ALL TASK COMPLETED SUCCESFULLY